TalTech Virumaa College, Kohtla-Järve

Erasmus+ BIP 2026 · Development of a Practical XR Application Prototype

IVAR Lab Digital Twin

Technical report · 16 October 2026

An XR application for the smart-home stand in room VK1 406. The scene shows the room, reads live device states from Home Assistant, and sends commands back to the same devices. Source: repository RedTeam-BIP-2026, branch main.

Name	Role	Institution
Satvik Velpula	Team leader	Metropolia, Finland
Bohdan Kyrychenko	Scan, models, and logger	Thomas More, Belgium
Marvin Wachter	Quest VR setup	Hochschule Esslingen, Germany
Chakib Lachaal	Room model in Unity	Šiauliai State UAS, Lithuania
Baddar Hani Badr Abobakr	Room model in Unity	TalTech, Estonia
Džoanna Šarlote Akselrode	Documentation	LU Banku augstskola and RTU, Latvia

Contents

1 Summary

2 Objective and requirements

3 Devices and software

4 System architecture

5 Demonstrations and testing

6 Problems and limitations

7 Results and further development

A Running the application

1. Summary

The IVAR Lab Digital Twin is one Unity scene, Assets/Scenes/IvarLab.unity. The scene name comes from an earlier reference model of TalTech's IVAR laboratory. The room that was scanned and built is VK1 406: about 5.81 m by 2.75 m, ceiling 2.91 m.

Home Assistant is the only link to the hardware. The application reads states over its WebSocket API and sends commands over its REST API, sometimes through a small Python program on the PC. KNX equipment is mounted in the room. This application does not send KNX telegrams, because the group addresses on the stand were not set up. Each device is used through the integration Home Assistant already has for it: Matter, Zigbee, or Z-Wave.

The separate behaviours are fused into five demonstrations in one scene: the lamp, the door, temperature and the siren, the motion scene, and the door announcement. On 14 October the lamp, the door, the motion scene, and the thermostat setpoint were exercised on the live stand from VR. Warming the physical temperature sensor until the reading passed 25 °C turned the frient siren on. The model updates from the state Home Assistant reports back. A press is not drawn as a success before the device answers.

The application runs in the Unity Editor and as a PC build. A headset on Link or Air Link shows that Play session. A package that runs on the headset by itself is built separately and talks to a laptop bridge.

2. Objective and requirements

The stand already reports to Home Assistant. The problem is to see that room in XR and to operate the same devices from inside the model, without a second control protocol.

The brief asks for a 3D space, an XR environment, visible device states, basic interaction, a data-exchange layer on the local network, at least two or three working use cases, testing, and this short document. A real device connection is preferred when it can be done simply and safely.

Requirement	Result	Match
Simple 3D space with devices	Room rebuilt from a LiDAR scan. Device models for the switch, lamp, sensor, thermostat, beacon, shade, and strip.	Met
Unity 2022.3.76f1	Unity 2022.3.62f3, the same LTS line.	Different patch
VRTK v4, or equivalent	XR Interaction Toolkit 3.5.0 with OpenXR 1.17.1.	Equivalent
Visualisation of states	Bindings drive lights, colour, the shade, the door hinge, and the thermostat text from live states.	Met
XR interaction	Controller ray with grip or trigger. Mouse in the Game view. Keys 1–7 in the Editor.	Met
Real connection, simulation, or a mix	Live connection. If Home Assistant is not connected, the press is ignored. The scene does not invent a state.	Met
At least 2–3 use cases	Five demonstrations in one scene: lamp, door, temperature and siren, motion scene, and door announcement.	Met
Testing	Dated tests on 12 and 14 October. The door announcement was not its own live test. Headset frame rate was not measured.	Partly met
Data-exchange layer	Home Assistant WebSocket for states. REST, or a local Python program, for commands.	Met
KNX smart-home lab	Devices are reached through Matter, Zigbee, or Z-Wave. See section 4.3.	Deviation
Sensors movable at runtime	Devices stay where the scan placed them.	Open

Table 1. The brief, and what this repository delivers on branch main.

3. Devices and software

The PC that runs the scene joins the lab router, SSID RUT361_F473. Home Assistant is at http://homeassistant.local:8123 on that network. It is not reachable from eduroam. The address http://192.168.15.249:8123 does not answer. The PC needs Unity 2022.3.62f3, Python 3, and the websockets package. Blender must be installed for Unity to import Assets/Models/ivar-lab-taltech.blend. Device models that are FBX files import without Blender.

Component	What it is
Lab room	VK1 406. About 5.81 × 2.75 m, ceiling 2.91 m.
Stand	Lamp, door contact, motion sensor, floor-heating thermostat, roller shade, LED strip, siren, Nest speaker.
Gateways	Aqara hub and USB controllers. They speak Zigbee, Z-Wave, and Matter to Home Assistant.
KNX hardware	Present on the stand. No group addresses are configured, so the twin does not use it.
Home Assistant	homeassistant.local:8123, on the lab network only.
Headset package	RedLab Twin, com.tredlab.redlabtwin, version 0.2.1, Android version code 10. Minimum SDK 32, target SDK 34.

Table 2. Hardware and the shipped headset package.

Layer	Item
Engine	Unity 2022.3.62f3. Input uses the Input System package.
XR	XR Interaction Toolkit 3.5.0, OpenXR 1.17.1, XR Management 4.7.0.
Helpers	Python 3 and websockets. Ports 8765 (lamp), 8766 (temperature), 8767 (motion).
Backend	WebSocket /api/websocket for states. REST /api/services for commands.
Modelling	Blender imports the room file. Unity does not run the Blender generator scripts.
Quest build	Menu RedLab > Build Quest APK. The release key is stored outside the repository.

Table 3. Software stack on branch main.

Device	Home Assistant entity	What changes in the scene
Aqara Dimmer Switch H2, Matter	light.aqara_dimmer_switch_h2_2	Pendant lamp and the LED on the wall switch
Aqara Door and Window Sensor T1	binary_sensor.aqara_door_and_window_sensor_t1_door	Door leaf. The model has no window sash, so this sensor drives the door.
Aqara Motion Sensor P1	binary_sensor.motion_sensor	Detection cone and the sensor LED
Floor-heating thermostat	sensor.aqara_floor_heating_thermostat_temperature	Measured °C. This value is read-only.
Thermostat setpoint	climate.aqara_floor_heating_thermostat	+ and − write the target, inside 5–35 °C.
Aqara Roller Shade Driver E1	cover.aqara_roller_shade_driver_e1	Shade fabric. Full travel is 30 seconds.
Fibaro RGBW Controller 2	light.rgbw_controller_2	Colour of the LED strip
frient SMSZB-120 Siren	siren.frient_a_s_smszb_120_siren	Warning beacon flashes while the siren is on
Google Nest speaker	media_player.famnest2	Jazz in the motion scene. “Door is open” from the door script.

Table 4. Devices, entities, and the matching object in the scene.

4. System architecture

The twin talks only to Home Assistant. Home Assistant talks to the stand. A solid step in Figure 1 is a link the twin uses. KNX is present and unused.

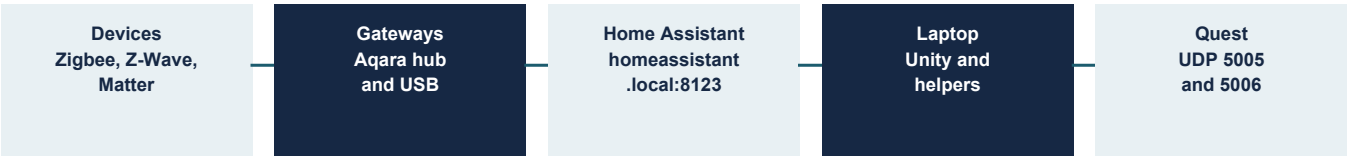


Figure 1. Every device is reached through Home Assistant. In the Editor the laptop speaks to Home Assistant directly. A standalone headset speaks only to twin_bridge.py on the laptop. Both join RUT361_F473.

4.1 Editor and PC

Figure 2 is the path when the scene runs in the Unity Editor, or as a PC build, on the same computer as the Python programs. States move down. Presses move back up.



Figure 2, states. A change arrives on the WebSocket and updates the matching object. The hub does not require a UDP packet on this path.



Figure 2, presses. The lamp, the setpoint, and the motion scene go through the local programs. The shade and the strip go straight to a Home Assistant service. door_announce.py is optional and is not started by Play.

A state reaches the model in five steps. The door is the example. The same steps apply to every device in Table 4.

- The sensor reports to Home Assistant.
- HomeAssistantClient is connected to ws://homeassistant.local:8123/api/websocket. It has authenticated, requested get_states, and subscribed to state_changed.
- The client keeps the message only if the entity is one of the bindings, or the speaker, and turns it into one twin.state.v1 message.
- On the Unity main thread the message is passed to TwinUdpReceiver. For this path that object is an in-memory hub.
- The matching binding updates the object. The door hinge rotates. A lamp enables its light. The shade spends 30 seconds travelling.

The token is entered in IVAR > Home Assistant Settings and stored on that computer, outside the project. It can also be supplied as HA_TOKEN. If the socket drops, the client tries again every 3 seconds. Save + Test is a separate HTTP check. A successful test does not open the Play-mode socket.

A controller ray, a mouse click, or a number key calls TwinCommandRouter. The router sends the command only when the WebSocket status is connected. Otherwise the press is discarded. The console logs “[Twin] ignored simulated … (real sensors only)”, and the panel says “Presses do nothing — Home Assistant not connected”. “Connecting…” is not “connected”. When the status is connected, the router sends the command and does not change the model itself. The model updates only when the new state comes back.

Press	Request	If that program is not running

Press	Request	If that program is not running
Motion sensor	POST 127.0.0.1:8767/scene	Ignored. The stand is unchanged.
Roller shade	cover.open_cover or close_cover	No second path
Lamp	POST 127.0.0.1:8765/devices/lamp/set	Unity calls light.turn_on or turn_off
LED strip	light.turn_on or turn_off	No second path
Thermostat + or −	POST 127.0.0.1:8766/devices/temperature/set	Unity calls climate.set_temperature
Door, measured °C, siren	No command. These report state.	—

Table 5. Where a press goes while Home Assistant is connected.

In the Editor, UDP port 5005 is an optional extra input from ha_twin_logger.py. Home Assistant never sends UDP. The panel line “Waiting for Home Assistant on UDP :5005” means the hub is empty. It does not mean the WebSocket is waiting for that port. On a standalone headset the same port is the return path in Figure 3.

4.2 Standalone headset

When the APK runs on the Quest, the headset holds no Home Assistant token. scripts/twin_bridge.py on the laptop holds the token, keeps the WebSocket open, and can start the three helper programs. The headset finds that laptop by a UDP broadcast to port 5006 every 2 seconds. No address is typed on the headset. The bridge returns states to the headset’s UDP port 5005, sends a heartbeat every 2 seconds, and carries each press back. Shade and strip presses become a Home Assistant service call. Lamp, setpoint, and motion presses become HTTP to 127.0.0.1 on the laptop, ports 8765, 8766, and 8767. A headset that stops sending hello is forgotten after 15 seconds. In the Editor this bridge client stays off.

Opening the project and pressing Play is how the application runs on a computer. A headset on Link or Air Link shows that same session, so the Python programs on 127.0.0.1 still apply. A package that runs on the headset alone is built once per release with RedLab > Build Quest APK (release). Close the Project Settings window before that build, otherwise Unity clears the signing passwords. People who only install an existing headset build do not run that menu.

4.3 Why KNX is not the data path

The brief describes a KNX lab. No twin data passes through KNX. The group addresses on the stand were not configured, so there is no address to write. Each live device is used through the integration Home Assistant already has. The lamp is the Matter dimmer in Table 4. The door is the Zigbee contact, because a Z-Wave contact on the stand did not answer. use_case_1/translate.py can explain a KNX telegram for teaching. The running application does not send one.

5. Demonstrations and testing

The application does not run the behaviours as separate programs in the scene. They are fused into five demonstrations.

Demonstration	Direction	Live result
1 Lamp	XR to the stand	Passed, 14 October
2 Door sensor	Stand to XR	Passed. Logger on 12 October. Twin on 14 October, about 1–2 seconds behind the sensor.
3 Temperature, setpoint, siren	Both	Setpoint passed, 14 October (33.0 to 32.5 °C). Siren confirmed by warming the sensor past 25 °C.
4 Motion, shade, strip, music	Stand, also started from XR	Passed, 14 October

Demonstration	Direction	Live result
Door announcement	Stand to the speaker and the scene	Not recorded as its own live test

Table 6. The five demonstrations and what was confirmed on the stand.

Lamp

A press calls use_case_1/lamp_api.py on port 8765. The script sends light.turn_on or light.turn_off for light.aqara_dimmer_switch_h2_2. Brightness is an integer from 0 to 255 on the same call. If the program is not running, Unity calls Home Assistant itself. On 14 October a VR press turned the physical lamp on, and changing the physical lamp updated the model.

Door

The Aqara T1 contact swings the door leaf by about 98 degrees. The model has no moving window sash, so this sensor drives the door. The logger recorded live open and close events on 12 October. The model followed the door on 14 October, about 1–2 seconds behind the sensor.

Temperature and alarm

The screen shows the measured temperature. The + and − buttons write the setpoint through use_case_3/temperature.py, inside 5–35 °C. Every 5 seconds the script reads Home Assistant. The alarm is on when the measured temperature is above 25 °C, or when the setpoint itself is above 25 °C. While it is on, the script turns the frient siren on, turns the dimmer off, and sets the strip to solid red. When the alarm ends, the siren goes off and those lights return to the state they had before. The beacon flashes from the siren state. The measured temperature cannot be written. On 14 October the “−” button moved the real setpoint from 33.0 °C to 32.5 °C. Warming the physical sensor by blowing on it raises the reading above 25 °C, and the siren on the stand turns on.

Motion, shade, strip, and music

scripts/motion_demo.py watches Home Assistant and does not edit its configuration. Motion on closes the shade. After 45 seconds without motion the script opens it. The wait is longer than the motor’s 30-second travel, so the shade can finish closing before it is told to open. A closed shade turns the strip red and plays Radio Swiss Jazz on the Nest speaker. An open or moving shade turns the strip green and stops the music. Pressing the motion sensor in VR posts to port 8767. Stopping the script restores the strip and the shade. On 14 October a VR shade press closed the real shade, and the motion scene ran with the strip red and the speaker playing. The scene still contains a local copy of this sequence with a 10-second wait. Real sensors only is on, so that copy is not applied.

Door announcement

scripts/door_announce.py watches the same door sensor. Each time it opens, Home Assistant speaks “Door is open” on the Nest speaker, at most once every 10 seconds. In the scene, a label and a recorded clip play as well. This speech path was not recorded as its own live test.

In the Editor the number keys are 1 door, 2 motion, 3 thermostat, 4 lamp, 5 beacon, 6 shade, 7 strip. The headset has no keyboard. The Quest package, version, and build script are in the project. Frame rate and latency on a headset were not measured.

6. Problems and limitations

Problem	Cause	Outcome

Problem	Cause	Outcome
KNX could not be commanded	Group addresses were not configured.	Devices are used through Home Assistant.
Home Assistant did not answer off the lab router	The server is only on RUT361_F473. Eduroam cannot resolve homeassistant.local.	The PC joins the lab router. The URL stays http://homeassistant.local:8123.
Play mode stayed on “connecting”	Save + Test does not open the Play-mode socket. A name lookup has no deadline.	The panel must show connected before a press is sent.
A local press fought the live test	A simulated toggle was applied while the real state was also arriving.	Real sensors only discards that toggle. The console logs ignored simulated.
The numeric Home Assistant address	http://192.168.15.249:8123 does not answer.	The name homeassistant.local is the address to use.

Table 7. Problems that changed how the twin is run.

Limitation	Effect
Devices cannot be moved at runtime	The request for flexible sensor placement remains open.
The siren cannot yet be acknowledged	It stays on until the reading and the setpoint are back at or below 25 °C.
Door announcement	The spoken line was not recorded as its own live test.
Headset performance	Frame rate and latency were not measured.
Optional UDP logger	Play mode does not need it. On the headset, port 5005 is instead the bridge return path.

Table 8. Limits that remain.

7. Results and further development

Branch main contains the scanned room, the device models, and one XR scene connected to the stand through Home Assistant. The lamp, the shade, the motion scene, and the thermostat setpoint send commands from XR to real devices. The door is the other direction: the real sensor moves the model. KNX, which the brief names, could not be used because the stand’s group addresses were not configured.

The project uses Unity 2022.3.62f3 rather than 2022.3.76f1, and the XR Interaction Toolkit rather than VRTK. Devices stay where the scan placed them.

Three items remain for a later version.

- The alarm can be acknowledged. A person in the room needs a way to silence the siren while the temperature is still above 25 °C, so the sound means the alarm was noticed and then stops.
- Each device needs a state label beside the object, so a change such as on, off, opening, or a new temperature is readable at the component as well as on the status panel.
- The helper programs should be able to address several Home Assistant devices from one condition, so heat, an open door, or motion can drive lights, shade, climate, and the siren together.

A. Running the application

- Join RUT361_F473. Open http://homeassistant.local:8123 in a browser and confirm the page loads.
- Open the Unity project in 2022.3.62f3 and load Assets/Scenes/IvarLab.unity. If IVAR Lab shows as a missing prefab, install Blender and reimport Assets/Models/ivar-lab-taltech.blend.
- Open IVAR > Home Assistant Settings, paste a long-lived access token, and choose Save + Test. The URL stays http://homeassistant.local:8123. The report must list the entities in the scene.
- Copy use_case_1/.env.example to use_case_1/.env. Set HA_URL and HA_TOKEN to the same values. The temperature program reads this file.
- From the repository root, start the three programs and leave the terminals open: python

use_case_1/lamp_api.py serve; python use_case_3/temperature.py serve; python scripts/motion_demo.py --url http://homeassistant.local:8123 --http 8767. On Windows, python -X utf8 avoids console errors on non-ASCII names.

- Optional: python scripts/door_announce.py --url <HA_URL> and python scripts/ha_twin_logger.py --url <HA_URL> --udp 127.0.0.1:5005. For a standalone headset, one window can replace the three programs: python scripts/twin_bridge.py --url http://homeassistant.local:8123. The headset and the laptop must be on the same network.
- Press Play. The panel must show Home Assistant: connected and Presses → real stand, and the three helpers as on. A console line “[Twin] ignored simulated” means the WebSocket is not connected and the stand was not commanded.
- Operate a device with the controller ray, with the mouse in the Game view, or with keys 1–7.

A headset package is built with RedLab > Build Quest APK (release) after Android Build Support is installed. Close the Project Settings window before that build. The release key is not in the repository.